Introduction to If-else in Java

1. If - Else

The if-else statement is used to make decisions in your program. It allows you to execute different blocks of code based on whether a condition is true or false.

Syntax:

```java
if (condition) {
    // Code to execute if condition is true
} else {
    // Code to execute if condition is false
}
```

Example:

```java
int age = 18;

if (age >= 18) {
    System.out.println("You are eligible to vote.");
} else {
    System.out.println("You are not eligible to vote.");
}
```

Output: You are eligible to vote.

Key Points:

- The condition must be a boolean expression (true or false)
- If the condition is true, the code inside the if block executes
- If the condition is false, the code inside the else block executes
- The else part is optional

2. If - else if - else

When you need to check multiple conditions, you can use the if-else if-else statement. This allows you to test several conditions in sequence.

Syntax:

```java
if (condition1) {
    // Code to execute if condition1 is true
} else if (condition2) {
    // Code to execute if condition2 is true
} else if (condition3) {
    // Code to execute if condition3 is true
} else {
    // Code to execute if all conditions are false
}
```

Example:

```java
int marks = 85;

if (marks >= 90) {
    System.out.println("Grade: A+");
} else if (marks >= 80) {
    System.out.println("Grade: A");
} else if (marks >= 70) {
    System.out.println("Grade: B");
} else if (marks >= 60) {
    System.out.println("Grade: C");
} else {
    System.out.println("Grade: F");
}
```

Output: Grade: A

Key Points:

- You can have multiple else if statements
- Only the first true condition will execute
- The final else is optional but recommended for handling all other cases

3. Nested if-else

Nested if-else means placing one if-else statement inside another if-else statement. This is useful when you need to check multiple conditions that depend on each other.

Syntax:

```java
if (outerCondition) {
    if (innerCondition) {
        // Code if both conditions are true
    } else {
        // Code if outer is true but inner is false
    }
} else {
    // Code if outer condition is false
}
```

Example:

```java
int age = 20;
boolean hasLicense = true;

if (age >= 18) {
    if (hasLicense) {
        System.out.println("You can drive a car.");
    } else {
        System.out.println("You need to get a license first.");
    }
} else {
    System.out.println("You are too young to drive.");
}
```

Output: You can drive a car.

Key Points:

- The inner if-else only executes if the outer condition is true
- You can nest as many levels as needed, but keep it readable
- Each if statement needs its own condition

4. If { if – else } – else { if – else }

This structure involves having complete if-else statements in both the if and else blocks of an outer if-else statement. Both branches contain their own decision-making logic.

Syntax:

```java
if (mainCondition) {
    if (condition1) {
        // Code block 1
    } else {
        // Code block 2
    }
} else {
    if (condition2) {
        // Code block 3
    } else {
        // Code block 4
    }
}
```

Example:

```java
boolean isWeekend = true;
boolean isRaining = false;

if (isWeekend) {
    if (isRaining) {
        System.out.println("Stay home and watch movies.");
    } else {
        System.out.println("Go to the park and enjoy the day.");
    }
} else {
    if (isRaining) {
        System.out.println("Take an umbrella to work.");
    } else {
        System.out.println("Have a great day at work.");
    }
}
```

Output: Go to the park and enjoy the day.

Key Points:

- Both the if and else blocks contain complete if-else statements
- This creates four possible execution paths
- Useful when you have two main scenarios, each with their own sub-decisions

---

Quiz Questions

1. What will be the output of the following code?

```java
int x = 10;
if (x > 5) {
    System.out.println("Hello");
} else {
    System.out.println("World");
}
```

a) Hello
b) World
c) Hello World
d) No output

Answer: a) Hello

2. In an if-else if-else statement, how many blocks of code can execute? a) All blocks
b) Only one block
c) At least two blocks
d) It depends on the conditions

Answer: b) Only one block

3. What is the correct syntax for an if statement in Java? a) if x > 5 { }
b) if (x > 5) { }
c) if [x > 5] { }
d) if {x > 5} ( )

Answer: b) if (x > 5) { }

4. What will happen if all conditions in an if-else if-else statement are false? a) Compilation error
b) Runtime error

c) The else block executes
d) Nothing happens

Answer: c) The else block executes

5. In nested if-else, the inner if statement executes only when: a) The outer condition is false
b) The outer condition is true
c) Both conditions are false
d) The inner condition is true

Answer: b) The outer condition is true

Practice Coding Questions

1. Write a Java program that takes a number as input and checks if it is positive, negative, or zero. Display appropriate messages for each case.

2. Create a program that determines the type of triangle based on three side lengths entered by the user (equilateral, isosceles, or scalene). Use nested if-else statements.

3. Write a program that simulates a simple calculator. Take two numbers and an operator (+, -, *, /) as input, then perform the operation and display the result. Handle division by zero.

4. Develop a program that determines if a student passes or fails based on marks in three subjects. The student passes if the average is >= 60 AND no individual subject has marks < 40. Use appropriate if-else structures.

5. Create a program that categorizes a person based on age: Child (0-12), Teenager (13-19), Adult (20-59), Senior (60+). Also, within the Adult category, distinguish between Young Adult (20-35) and Middle-aged (36-59).